MEDICAL VISUAL QUESTION ANSWERING

Dissertation Submitted by

SWATHY T E
(**Reg. No:MG24C7115007**)

under the guidance of

Dr. Ansuman Mahapatra
Associate Professor
Computer Science & Engineering
National Institute of Technology
Puducherry, Karaikal

&

Mr. JIBY JOSEPH
Assistant Professor
School of Data Analytics
Mahatma Gandhi University – Kottayam

in partial fulfilment for the award of the degree of

MASTER OF SCIENCE
In
DATA SCIENCE AND ANALYTICS

SCHOOL OF DATA ANALYTICS
MAHATMA GANDHI UNIVERSITY KOTTAYAM
June, 2026

राष्ट्रीय प्रौद्योगिकी संस्थान पुदुच्चेरी
(शिक्षा मंत्रालय के तहत राष्ट्रीय महत्व का एक संस्थान,भारत सरकार)
तिरुवेट्टाकुड़ी, कारैक्काल - 609 609, केंद्र शासित प्रदेश,पुदुच्चेरी
NATIONAL INSTITUTE OF TECHNOLOGY PUDUCHERRY
(An Institution of National Importance under Ministry of Education, Govt of India)
Thiruvettakudy, Karaikal – 609 609, Union Territory of Puducherry

16/06/2026

CERTIFICATE OF INTERNSHIP

This is to certify that **Ms. Swathy T E (MG24C75007)**, II-year M.Sc. student in the Department of Data Science and Data Analytics at the Mahatma Gandhi University, Kottayam has successfully completed her full-time internship titled ***"Medical Visual Question Answering."*** The internship was carried out from 01/02/2026 to 29/05/2026 under the supervision of Dr. Ansuman Mahapatra, Associate Professor in the Department of Computer Science and Engineering at the National Institute of Technology Puducherry, Karaikal.

16/06/2026

Dr. Ansuman Mahapatra
Associate Professor
Department of CSE
NIT Puducherry

16/06/2026

Dr. Surendiran B
Professor-In-Charge
Training & Placement
NIT Puducherry

PROFESSOR IN-CHARGE
TRAINING AND PLACEMENT
National Institute of Technology Puducherry
Thiruvettakudy, Karaikal - 609 609
Puducherry U.T.

Phone / दूर भाष : 04368-265233 E-mail / ई-मेल : placement@nitpy.ac.in
Web / वेब : www.nitpy.ac.in

CERTIFICATE

This is to certify that dissertation entitled **“MEDICAL VISUAL QUESTION ANSWERING”,** is an original and authentic record of study carried out by **SWATHY T E (Reg. No: MG24C7115007),** under my guidance and supervision, in partial fulfilment of the requirement for the award of the degree of Master of Science, during the academic year 2024-26 and that it has not been previously submitted to the award of any degree, diploma, fellowship or any other similar recognition.

Mr. JIBY JOSEPH
Project Guide

Dr.K K JOSE
Director

DECLARATION

I **SWATHY T E (Reg. No:MG24C7115007)**, hereby declare that I have completed my dissertation work entitled **" MEDICAL VISUAL QUESTION ANSWERING"** during the period of February to May 2026, in partial fulfillment of MSc Data Science And Data Analytics course under the guidance of **Dr. Ansuman Mahapatra**. The matter embodied in this project work has not been submitted earlier for the award of any degree to the best of my knowledge and belief.

SWATHY T E
Kottayam
M.Sc.Data Science
School of Data Analytics
Mahatma Gandhi University
Kottayam

Countersigned by
Dr. K. K. Jose
Director
School of Data Science
And Data Analytics

ACKNOWLEDGEMENT

At the outset, I thank God Almighty for making this endeavour a success. The success and final outcome of this project required a lot of guidance and assistance from many people and I am extremely fortunate to have this support throughout the completion of my project work.I express my deep sense of gratitude to the Director, School of Data Analytics, for the constant support and encouragement. I owe my profound gratitude to my project guide **Dr. Ansuman Mahapatra**, NATIONAL INSTITUTE OF TECHNOLOGY PONDICHERRY who took keen interest in my project work and guided me throughout by providing all the necessary information for developing a good project.I would also like to thank all the faculty members of the School of Data Analytics, Mahatma Gandhi University, for their valuable guidance and suggestions. I am grateful to the developers and researchers behind Google's MedGemma model and the Radiology AI community for making the RadImageNet-VQA dataset publicly available. I extend my sincere thanks to the Hugging Face community for providing the infrastructure and tools that made this research possible.Finally, I wish to thank my friends and family who have helped me directly and indirectly in the successful completion of this project work.

SWATHY T E

ABSTRACT

Medical Visual Question Answering (Med-VQA) is a challenging multimodal task that requires deep understanding of medical images combined with natural language reasoning. This project presents a complete pipeline for fine-tuning, validating, and evaluating Google's MedGemma-4B-IT model on the RadImageNet-VQA dataset — a radiology-focused visual question answering benchmark available as raidium/RadImageNet-VQA on the Hugging Face Hub.

The project involves four key phases: (1) custom reasoning-augmented dataset creation from the RadImageNet-VQA dataset, (2) supervised fine-tuning of the MedGemma-4B-IT model using LoRA (Low-Rank Adaptation) for parameter-efficient adaptation, (3) model validation on held-out data to track loss, perplexity and accuracy convergence, and (4) comprehensive evaluation of the base (pre-trained) model versus the LoRA fine-tuned model using ROUGE, BLEU, Exact Match and F1 metrics on both the reasoning text and the final answer.

The LoRA fine-tuned MedGemma model demonstrates a substantial improvement over the base pre-trained model across every evaluation metric. Reasoning-augmented training data with chain-of-thought style answer generation enhanced the model's ability to provide clinically meaningful, well-justified responses. Training and validation curves (loss, learning rate, gradient norm, perplexity and accuracy) confirmed stable convergence without overfitting.

The results show that LoRA-based domain-specific fine-tuning with high-quality reasoning-augmented datasets substantially improves performance on radiology visual question answering tasks, contributing to the broader goal of AI-assisted medical image analysis and clinical decision support systems.

CONTENTS

CHAPTER 1

INTRODUCTION

Artificial Intelligence has witnessed transformative growth in multimodal learning, enabling systems to understand and reason over both visual and textual information simultaneously. Among the most impactful applications of this progress is Medical Visual Question Answering (Med-VQA), a task that combines computer vision with natural language processing to answer clinically relevant questions about medical images. Med-VQA holds enormous potential in healthcare, as it can assist radiologists, clinicians, and researchers in extracting precise information from complex medical imagery such as X-rays, CT scans, MRIs, and histopathology slides.

The global burden of medical imaging interpretation is immense. Radiological imaging produces millions of images annually worldwide, and the shortage of trained radiologists — especially in low- and middle-income countries — creates significant delays in diagnosis. Automated Med-VQA systems can act as intelligent assistants, answering questions about anatomical structures, pathological findings, imaging modalities, and diagnostic features, thereby accelerating clinical workflows and potentially improving patient outcomes.

Traditional approaches to visual question answering in the medical domain have been constrained by limited dataset availability, lack of reasoning transparency, and the complexity of medical imaging. Early systems relied on simple classification-based pipelines that could answer only closed-ended questions (e.g., yes/no or single-word answers). These methods failed to generalize well to open-ended, clinically meaningful queries that require understanding anatomical context, disease characteristics, or imaging artifacts.

The emergence of large Vision-Language Models (VLMs) has dramatically changed this landscape. Models such as CLIP, BLIP, LLaVA, and Google's Gemma family have demonstrated remarkable capabilities in zero-shot and few-shot visual reasoning. However, medical domains present unique challenges that require domain-specific adaptation. General-purpose VLMs lack the specialized knowledge of medical terminology, anatomical structures, imaging modalities, and pathological features necessary for accurate clinical question answering.

This project addresses these challenges by fine-tuning Google's MedGemma-4B-IT model — a medical-domain adapted vision-language model — on the RadImageNet-VQA dataset. The project involves building a complete end-to-end pipeline: from reasoning-augmented dataset creation to fine-tuning, validation, and rigorous evaluation against baseline and competing models. The goal is to develop a specialized Med-VQA system that can accurately, reliably, and interpretably answer questions about radiology images.

1.1 PROBLEM STATEMENT

Medical imaging plays a central role in modern diagnosis, yet the interpretation of radiological images remains a time-intensive, expertise-demanding task performed exclusively by trained professionals. The growing volume of medical images combined with the global shortage of radiologists creates a critical bottleneck in healthcare delivery. Furthermore, existing automated systems for medical image analysis are predominantly limited to specific classification tasks and cannot engage in natural language-based question answering — a mode of interaction that is most natural for clinical workflows.

Current challenges in Medical Visual Question Answering include:

- Scarcity of large-scale, high-quality annotated medical VQA datasets covering diverse imaging modalities.
- Limited reasoning transparency — most models produce answers without explanations, making them unsuitable for clinical trust.
- Domain shift between general-purpose VLMs trained on natural images and the visual semantics of radiology images.
- Lack of fine-grained evaluation benchmarks that assess clinical relevance alongside linguistic quality.
- Computational constraints in deploying large multimodal models in clinical or research settings.

To address these challenges, there is a need for a domain-specifically fine-tuned vision-language model capable of providing accurate, reasoned answers to radiology-focused questions. This

project aims to develop such a system by combining state-of-the-art parameter-efficient fine-tuning techniques with a carefully constructed reasoning-augmented dataset.

1.2 OBJECTIVES

The primary objectives of this project are:

- To construct a reasoning-augmented Medical VQA dataset from the raidium/RadImageNet-VQA benchmark by generating chain-of-thought style answers that provide clinical context.
- To fine-tune Google's MedGemma-4B-IT vision-language model using LoRA (Low-Rank Adaptation) for memory-efficient, domain-specific adaptation.
- To validate the fine-tuned model against a held-out validation set and monitor training dynamics including loss convergence.
- To rigorously evaluate the fine-tuned model against baseline models (pre-trained MedGemma) and competing Med-VQA systems using standard evaluation metrics: Exact Match, BLEU, METEOR, and cosine similarity.
- To analyze the qualitative and quantitative improvements achieved through fine-tuning and reasoning augmentation.
- To demonstrate a complete, reproducible pipeline for domain-adaptive fine-tuning of large vision-language models in the medical imaging domain.

CHAPTER 2

LITERATURE REVIEW

The field of Visual Question Answering (VQA) was introduced by Antol et al. [1] as a benchmark task requiring systems to answer free-form natural language questions about images. While early VQA systems focused on general-domain images (COCO, Visual Genome), the unique characteristics of medical imaging prompted the development of specialized Med-VQA benchmarks and models.

Lau et al. [2] introduced the VQA-RAD dataset — one of the first manually curated Med-VQA datasets covering radiology images with both open-ended and closed-ended questions. Their study established baseline performance using attention-based VQA models and highlighted the difficulty of open-ended medical question answering compared to binary classification tasks. The VQA-RAD dataset has since become a standard benchmark for evaluating Med-VQA systems.

Ben Abacha et al. [3] proposed the ImageCLEF VQA-Med challenge, providing large-scale annotated datasets for medical VQA. Their work demonstrated that category-aware question routing — separating questions by type (abnormality, modality, organ) — significantly improved system performance. This influenced subsequent work on multi-task learning for Med-VQA.

Li et al. [4] developed LLaVA-Med, a medical adaptation of the LLaVA (Large Language and Vision Assistant) model, by instruction-tuning on biomedical figure-caption pairs collected from PubMed. LLaVA-Med demonstrated strong zero-shot performance on multiple Med-VQA benchmarks and showed that instruction-following large language models can be effectively adapted to medical visual reasoning through curated conversational fine-tuning.

Zhang et al. [5] introduced BioMedCLIP, a contrastive vision-language model trained on 15 million biomedical image-text pairs from PubMed Central. BioMedCLIP demonstrated the importance of domain-specific pre-training for medical image understanding, motivating the use of medically pre-trained backbones such as MedGemma in the present work

Hu et al. [6] introduced LoRA (Low-Rank Adaptation) for efficient fine-tuning of large language models, demonstrating that trainable low-rank decomposition matrices injected into transformer layers could match full fine-tuning performance with significantly fewer trainable parameters. This

parameter-efficient fine-tuning strategy is directly adopted in this project to adapt MedGemma-4B-IT to the radiology VQA domain.

Google's release of the Gemma and MedGemma [7] model families marked a significant advancement in open-weight medical AI. MedGemma-4B-IT incorporates a SigLIP vision encoder with the Gemma-3 language backbone, pre-trained on diverse medical imaging data. Unlike general-purpose VLMs, MedGemma has demonstrated strong out-of-the-box performance on medical benchmarks, making it an ideal foundation for domain-specific fine-tuning.

The literature collectively establishes that: (1) domain-specific fine-tuning on curated medical datasets substantially improves Med-VQA performance over zero-shot baselines; (2) reasoning-augmented training data with chain-of-thought explanations improves the interpretability and accuracy of model responses; and (3) parameter-efficient fine-tuning methods such as LoRA [6] make large-model fine-tuning computationally feasible. This project builds upon these insights to develop an improved radiology-focused VQA system.

CHAPTER 3

METHODOLOGY

This project employs a systematic, multi-stage methodology that covers dataset creation, model selection, fine-tuning strategy, and evaluation protocol. The pipeline is designed to be end-to-end reproducible and leverages LoRA (Low-Rank Adaptation), a parameter-efficient fine-tuning technique, to adapt a large medical vision-language model to the specific task of radiology visual question answering. The complete pipeline, from raw dataset to the final evaluated model, is illustratedFigure3.1.

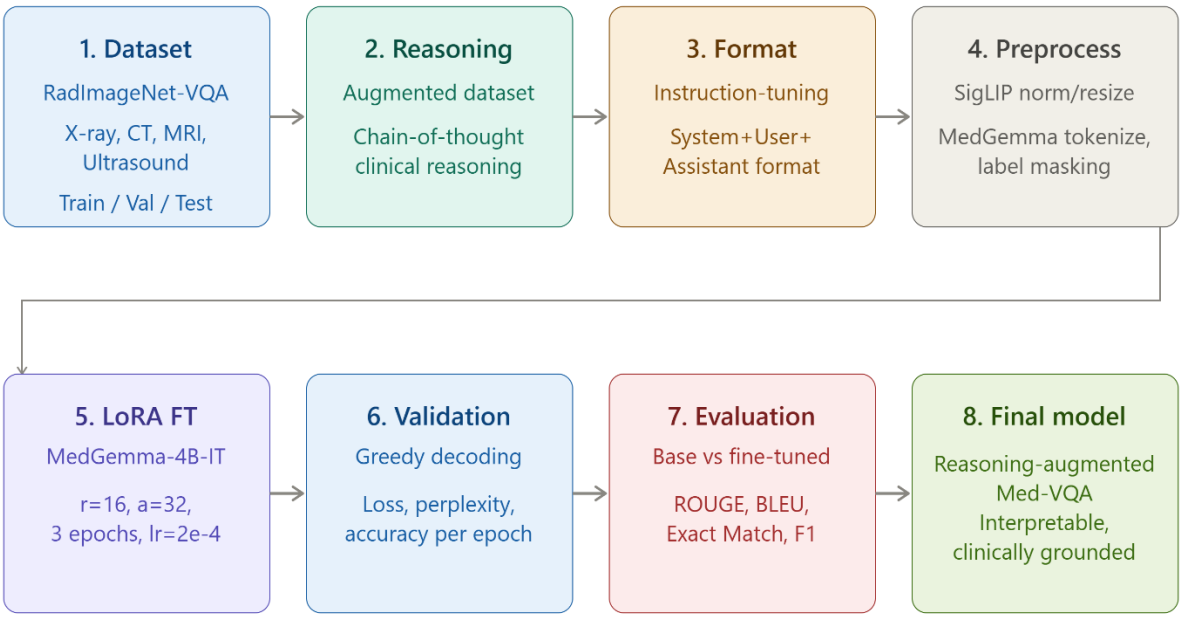


Figure 3.1: Full pipeline methodology — from RadImageNet-VQA dataset to LoRA fine-tuned MedGemma-4B-IT Med-VQA model, covering reasoning-augmented dataset creation, preprocessing, LoRA fine-tuning, validation and evaluation.

3.1 DATASET: RadImageNet-VQA

The primary dataset used in this project is the raidium/RadImageNet-VQA dataset, available on the Hugging Face Hub. RadImageNet-VQA[10] is a radiology-focused visual question answering dataset derived from the RadImageNet collection of medical imaging data. The dataset provides

question-answer pairs grounded in radiology images spanning multiple modalities including X-ray, CT, and MRI.

Dataset Characteristics

Attribute	Details
Source	raidium/RadImageNet-VQA (Hugging Face)
Image Modalities	X-ray, CT, MRI, Ultrasound
Question Types	Open-ended, Closed-ended, Descriptive
Answer Format	Free-text natural language
Dataset Split	Train / Validation / Test
Image Format	JPEG/PNG, variable resolution
Primary Domain	Radiology & Medical Imaging

Table 3.1: RadImageNet-VQA Dataset Characteristics — Analysis.

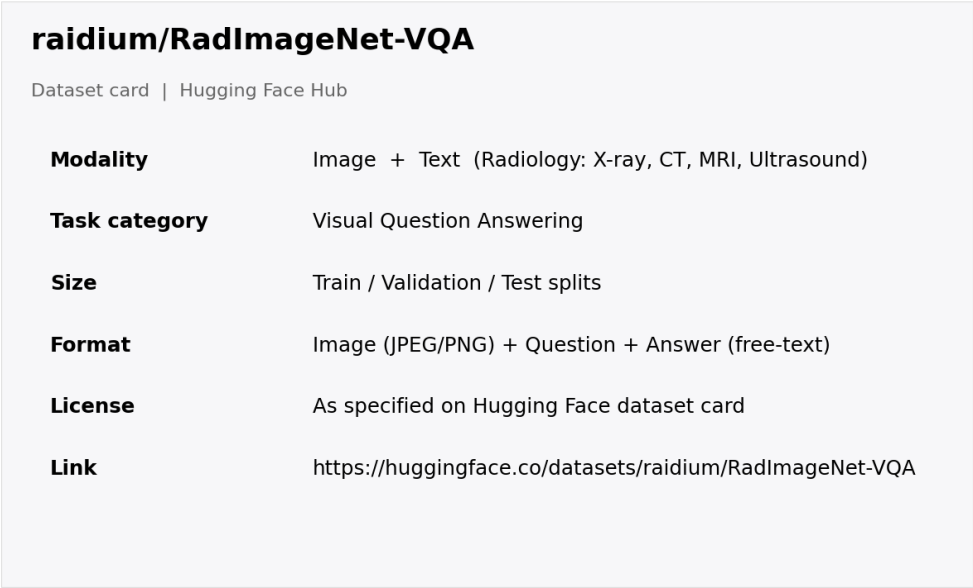
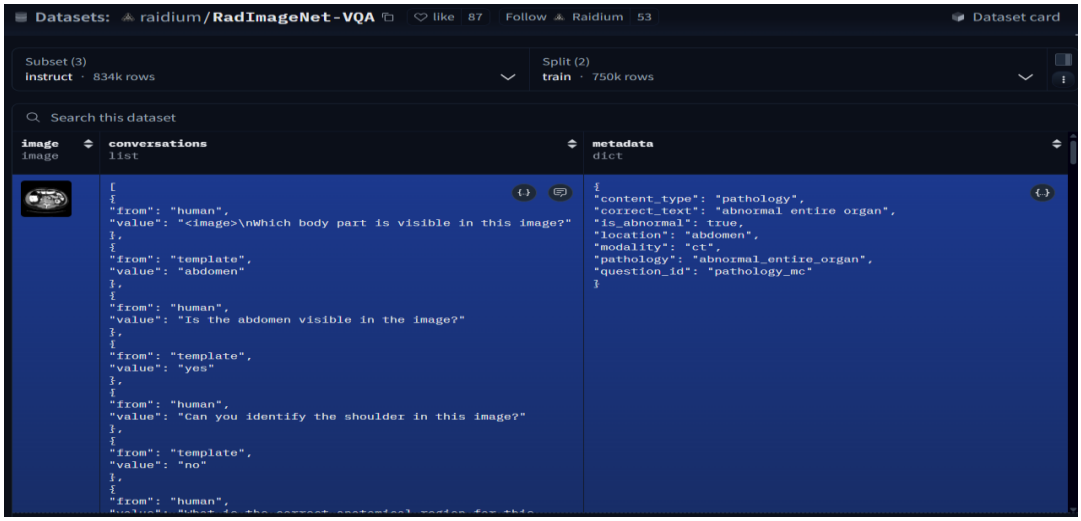


Figure 3.2: Dataset card / characteristics overview of raidium/RadImageNet-VQA on Hugging Face (illustrative example)

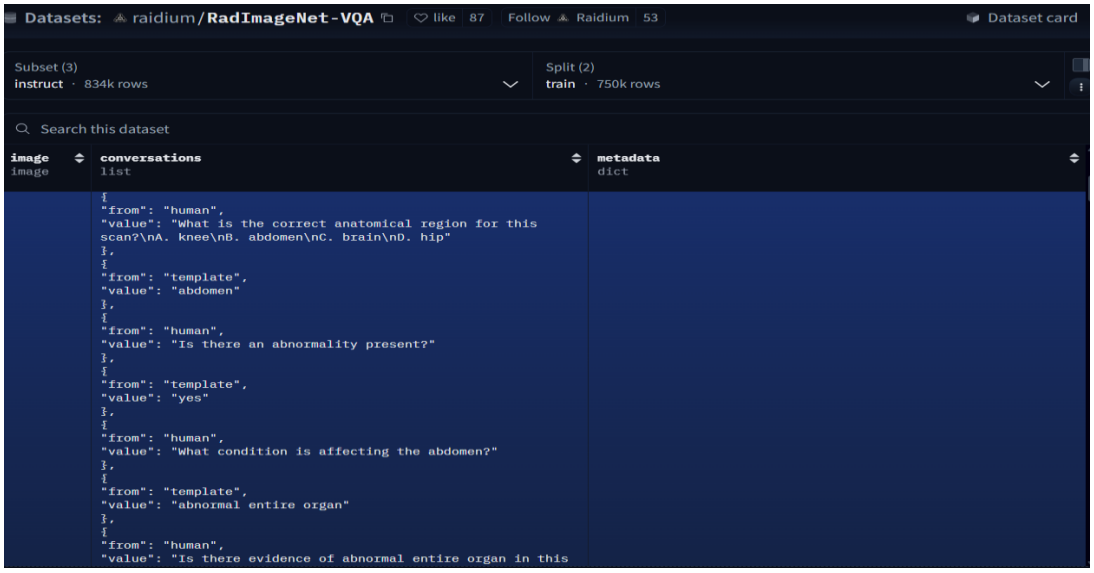
Figure 3.2: Sample Records from the RadImageNet-VQA Dataset (raidium/RadImageNet-VQA)

The below illustrate three consecutive panels from the Hugging Face dataset viewer for raidium/RadImageNet-VQA (instruct subset, train split, 750k rows). Together they show a

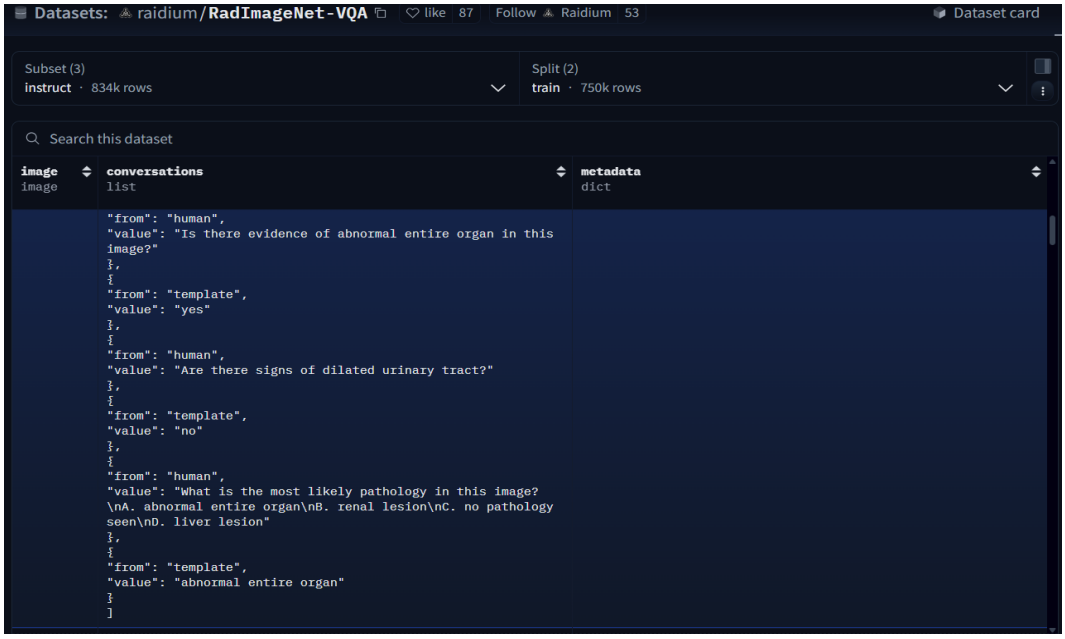
complete sample: a CT image of the abdomen (rendered as a thumbnail in column “image”), its full multi-turn conversation list (column “conversations”), and its metadata dictionary (column “metadata”). The conversation follows a human–template dialogue structure in which human turns pose questions and template turns supply ground-truth short answers. This structured format is converted into reasoning-augmented instruction-tuning samples for LoRA fine-tuning of MedGemma-4B-IT.



(a) RadImageNet-VQA dataset viewer showing the instruct subset with 834k rows — each row contains an image, a conversations list of multi-turn Q&A pairs, and a metadata dictionary with fields such as content_type, correct_text, is_abnormal, location, modality, pathology and question_id.



(b) Continuation of a sample conversation: the template answers anatomical region (“abdomen”), abnormality presence (“yes”), condition name (“abnormal entire organ”), and binary verification questions for specific pathologies such as dilated urinary tract.



(c) Final turns of the same sample conversation: binary confirmation of the pathological finding and a multiple-choice pathology question whose template answer is "abnormal entire organ", illustrating the structured conversational format used for LoRA fine-tuning.

Reasoning-Augmented Dataset Creation

A key contribution of this project is the construction of a reasoning-augmented version of the RadImageNet-VQA dataset using the create_reasoning_vqa_dataset pipeline. Standard VQA datasets provide only short, direct answers that lack clinical reasoning context. To address this limitation, chain-of-thought style reasoning was added to each answer to help the model learn not just what the answer is, but why it is the correct answer.

The dataset creation process involves:

- Loading raw image-question-answer triplets from the RadImageNet-VQA dataset via the Hugging Face datasets library.
- Generating reasoning-augmented answers using a prompting strategy that elicits step-by-step clinical explanations for each question-answer pair.
- Formatting the augmented data into the instruction-tuning format expected by MedGemma-4B-IT: a multi-turn conversation format with system prompt, user query (including image), and assistant response.

- Splitting the augmented dataset into training & validation, with stratified sampling to maintain class balance.
- Serializing the final dataset to disk for use in subsequent fine-tuning and evaluation stages.

3.2 MODEL ARCHITECTURE: MedGemma-4B-IT

The base model used in this project is google/medgemma-4b-it — a 4-billion parameter multimodal instruction-tuned model developed by Google. MedGemma is part of the Gemma 3 model family and is specifically designed for medical imaging applications.

Architecture Overview

MedGemma-4B-IT consists of two primary components:

- Vision Encoder: A SigLIP (Sigmoid Loss for Language Image Pre-training) vision encoder that processes medical images into visual feature representations. SigLIP is particularly well-suited for medical imaging as it was trained with a sigmoid-based contrastive loss that handles varying image-text pair quality more robustly than standard CLIP.
- Language Model Backbone: A Gemma-3 instruction-tuned language model that processes the concatenated visual features and text tokens to generate natural language responses. The language backbone uses grouped-query attention and rotary positional embeddings for efficient long-context processing.

The model accepts multimodal inputs: a medical image is processed through the vision encoder to produce visual tokens, which are then concatenated with the tokenized text query and passed through the language model to generate a free-text answer. This architecture enables the model to ground its language generation in visual evidence, which is critical for accurate medical image interpretation.

Pre-training and Medical Adaptation

MedGemma was pre-trained on a diverse corpus of medical imaging data including radiology images with associated reports, pathology images with annotations, ophthalmic images, and biomedical literature. This medical pre-training gives MedGemma a strong foundation in medical

visual semantics and clinical terminology, making it significantly more suitable as a starting point for Med-VQA fine-tuning compared to general-purpose VLMs.

3.3 LORA FINE-TUNING STRATEGY

Fine-tuning a 4-billion parameter model with full parameter updates would require prohibitive GPU memory resources and risks catastrophic forgetting of the model's pre-trained medical knowledge. To make fine-tuning computationally tractable while preserving the base model's capabilities, this project employs LoRA (Low-Rank Adaptation), a parameter-efficient fine-tuning technique.

Low-Rank Adaptation (LoRA)

Rather than updating all model parameters, LoRA injects trainable low-rank decomposition matrices into selected linear layers of the transformer. For a weight matrix W of dimensions d x k, LoRA parameterizes the weight update as: delta_W = B x A, where A is a d x r matrix and B is a r x k matrix, with r << min(d, k). Only matrices A and B are trained, while the original W remains frozen. The hyperparameter r (the LoRA rank) controls the expressivity-efficiency tradeoff.

In this project, LoRA adapters are injected into the query, key, value, and output projection matrices of the self-attention layers, as well as the feed-forward network (FFN) layers. The LoRA configuration used is:

Parameter	Value
LoRA Rank (r)	16
LoRA Alpha	32
LoRA Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Quantization	4-bit NF4 with double quantization
Compute dtype	bfloat16
Trainable Parameters	~42M (out of ~4B total)

Table 3.1: LoRA Configuration Parameters — Analysis.

3.4 PYTHON LIBRARIES AND FRAMEWORKS

The following key libraries and frameworks were used in this project:

- Hugging Face Transformers: Core library for loading, configuring, and training the MedGemma model. Provides AutoModelForCausalLM, AutoProcessor, and Trainer utilities.
- Hugging Face PEFT (Parameter-Efficient Fine-Tuning): Provides LoraConfig and get_peft_model utilities for injecting LoRA adapters into the model.
- bitsandbytes: Provides 4-bit quantization (BitsAndBytesConfig) for loading the model with NF4 quantization and double quantization.
- Hugging Face datasets: For loading the RadImageNet-VQA dataset and managing custom dataset creation and streaming.
- Hugging Face TRL (Transformer Reinforcement Learning): Provides the SFTTrainer (Supervised Fine-Tuning Trainer) for instruction-tuning workflows.
- torch (PyTorch): Core deep learning framework for tensor operations, GPU management, and model training.
- PIL (Pillow): For medical image loading, resizing, and preprocessing before input to the vision encoder.
- evaluate (Hugging Face): For computing BLEU, METEOR, and other standard NLP evaluation metrics.
- sentence-transformers: For computing semantic cosine similarity between generated and reference answers.
- scikit-learn: For computing classification metrics and dataset splitting utilities.
- numpy & pandas: For numerical operations and data manipulation throughout the pipeline.

CHAPTER 4

EXPERIMENTAL ANALYSIS

This chapter describes the step-by-step implementation of the Medical Visual Question Answering pipeline, from dataset creation through LoRA-based model fine-tuning to evaluation. Each phase is described in detail with reference to the corresponding code components and to the training/validation graphs produced during the experiment.

4.1 DATASET CREATION AND REASONING AUGMENTATION

The first phase of the project involves creating the reasoning-augmented VQA dataset. The raw RadImageNet-VQA dataset provides basic question-answer pairs, but these short answers lack the clinical reasoning that helps a model learn the underlying medical knowledge. The create_reasoning_vqa_dataset.py script implements the full dataset creation pipeline.

Loading the Base Dataset

The RadImageNet-VQA dataset is loaded from Hugging Face using the datasets library. Each sample contains a medical image, a natural language question about the image, and a ground-truth answer. The dataset is loaded with streaming=False to enable full dataset processing and split management.

Reasoning-Augmented Answer Generation

For each question-answer pair, the dataset creation pipeline generates an extended reasoning-augmented answer. The reasoning augmentation follows a structured clinical reasoning template:

- Observation: What is directly visible in the image relevant to the question.
- Analysis: Clinical interpretation of the observed findings.
- Conclusion: The direct answer to the question with justification.

This chain-of-thought format encourages the model to learn structured clinical reasoning rather than pattern-matching to short answers. The augmented answers significantly increase the training signal per sample and produce a more interpretable model.

Instruction-Tuning Format

The augmented data is converted into the conversation format expected by MedGemma-4B-IT. Each training sample is structured as a multi-turn conversation with the following components:

- System message: Establishes the model's role as a medical imaging expert.
- User message: Contains the medical image (as a visual token) and the natural language question.
- Assistant message: Contains the reasoning-augmented answer.

The processor's apply_chat_template method is used to convert the conversation structure into model-ready input tokens, ensuring that the image tokens are correctly positioned and that the attention mask appropriately masks padding tokens.

4.2 DATA PREPROCESSING AND PREPARATION

Before training, the dataset undergoes several preprocessing steps to ensure compatibility with the MedGemma model:

Image Preprocessing

Medical images in the RadImageNet-VQA dataset come in varying sizes, formats, and modalities. The MedGemma processor handles image normalization, resizing, and conversion to the required tensor format. Images are converted to RGB if necessary (some grayscale radiology images require channel replication), resized to the model's expected input resolution, and normalized using the SigLIP vision encoder's normalization statistics.

Text Tokenization

Question and answer texts are tokenized using the MedGemma tokenizer. Special tokens for the beginning of sequence, end of sequence, image, and turn separators are correctly inserted by the apply_chat_template method. The maximum sequence length is set based on the model's context window, with longer sequences truncated and shorter sequences padded to the batch maximum.

Data Collation

A custom data collator is implemented to handle the multimodal nature of the training batches. The collator pads sequences to the same length within each batch, creates appropriate attention masks, and sets the labels for causal language modeling — masking the prompt tokens so that the loss is only computed on the answer tokens. This teacher-forcing training objective ensures the model learns to generate the complete reasoning-augmented answer conditioned on the image and question.

4.3 MODEL FINE-TUNING WITH LORA

The fine-tuning phase is implemented in the finetune_medgemma.py script and uses the Hugging Face TRL library's SFTTrainer for supervised instruction-tuning.

Model Loading

The MedGemma-4B-IT model is loaded in bfloat16 precision using AutoModelForCausalLM, with device_map='auto' to automatically distribute layers across available GPU devices. No quantization is applied — the base model weights remain in their original precision, and only the LoRA adapters introduced in the next step are trained.

LoRA Adapter Injection

After loading the quantized base model, LoRA adapters are injected using the PEFT library's LoraConfig and get_peft_model function. The prepare_model_for_kbit_training function is called first to cast layer norm weights to float32 and enable gradient checkpointing, which reduces activation memory at the cost of additional computation. The LoRA adapters are then injected into the target modules, adding approximately 42 million trainable parameters to the frozen 4-billion parameter base model.

Training Configuration

The SFTTrainer is configured with the following training hyperparameters:

Hyperparameter	Value
Number of Epochs	3
Per-device Train Batch Size	1

Gradient Accumulation Steps	8
Effective Batch Size	8
Learning Rate	2e-4
LR Scheduler	Cosine with warmup
Warmup Ratio	0.03
Optimizer	paged_adamw_32bit
Weight Decay	0.001
Max Gradient Norm (Clipping)	0.3
Mixed Precision	bf16
Logging Steps	10
Save Strategy	epoch

Table 4.1: Training Hyperparameter Configuration — Analysis.

Training Process

During training, the SFTTrainer iterates over the training dataset in mini-batches. For each batch, the forward pass computes the next-token prediction logits, the cross-entropy loss is computed only on the answer tokens (prompt tokens are masked with -100 in the labels), and the backward pass computes gradients with respect to the LoRA parameters. Gradient accumulation over 8 steps effectively simulates a larger batch size, stabilizing training. The paged_adamw_32bit optimizer manages optimizer states in paged memory, further reducing GPU memory pressure.

Training loss and validation loss are logged at every 10 steps, enabling real-time monitoring of convergence. The model checkpoint is saved at the end of each epoch for subsequent evaluation.

4.4 MODEL VALIDATION

The model validation phase evaluates the fine-tuned model on the held-out validation subset to assess generalization performance. After each training epoch, the fine-tuned model generates predictions for all samples in the validation set using greedy decoding (temperature=0, top-p=1.0). Training and validation dynamics were tracked across steps and epochs and are presented in the figures below

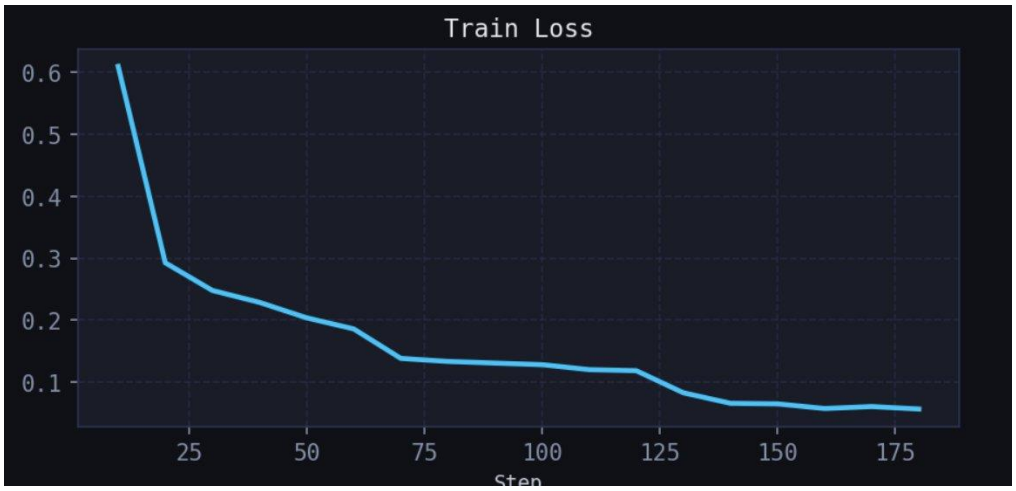


Figure 4.1: Training loss vs. training step. The loss drops sharply from approximately 0.61 in the first logged steps to below 0.10 by step ~130, then stabilizes around 0.05-0.06, indicating rapid and stable convergence of the LoRA adapters

Figure 4.2: Evaluation (validation) loss vs. epoch. The validation loss decreases from about 0.201 at epoch 1 to about 0.163 at epoch 2 and remains nearly flat at epoch 3, showing that the model generalizes well to unseen validation samples without overfitting.

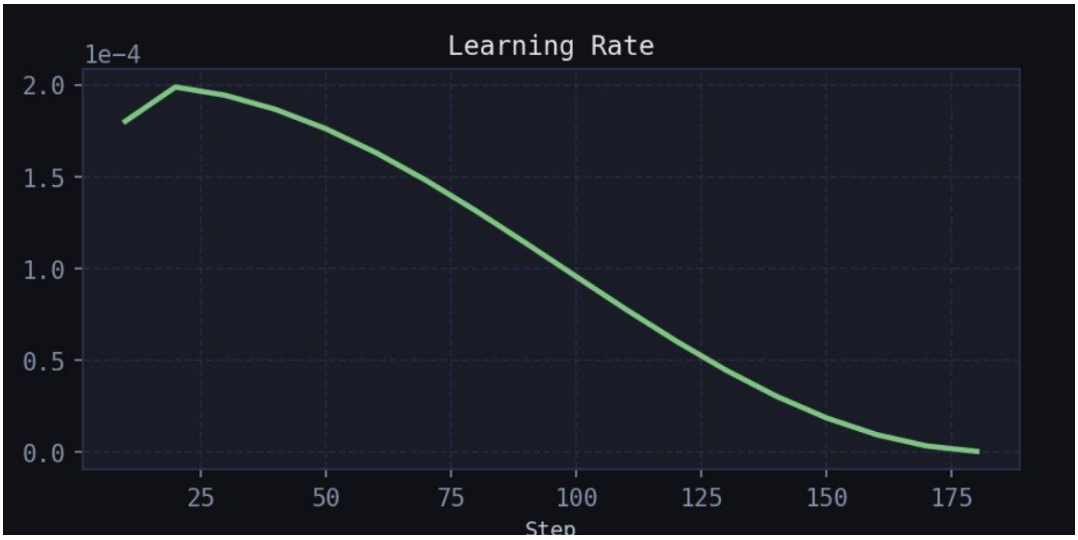


Figure 4.3: Learning rate schedule vs. training step. The cosine learning-rate schedule with warmup peaks at 2e-4 around step 20 and smoothly decays to near zero by the end of training, as configured in Table 4.1 (Training Configuration).

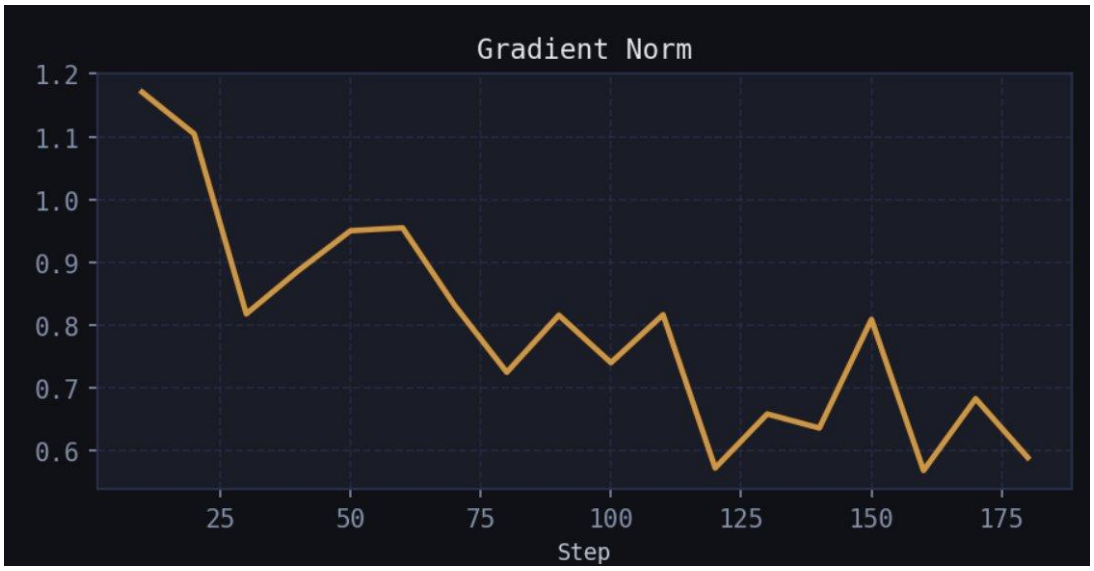


Figure 4.4: Gradient norm vs. training step. The gradient norm starts around 1.17, drops sharply within the first 30 steps, and then fluctuates within a stable band (roughly 0.55-0.95) for the remainder of training, indicating controlled and stable optimization under the 0.3 gradient-clipping threshold.

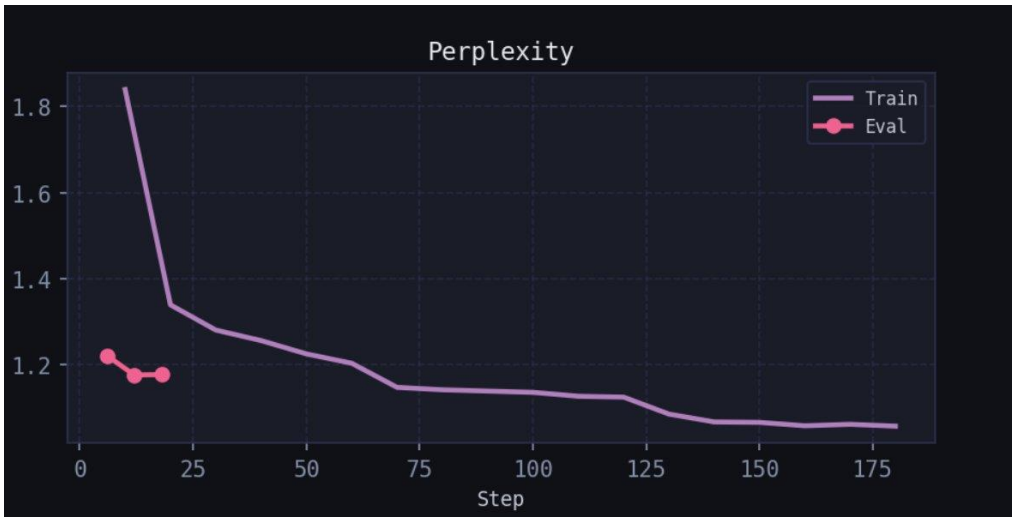


Figure 4.5: Perplexity (train and eval) vs. training step. Training perplexity falls steeply from about 1.84 to near 1.05 by the end of training, while evaluation perplexity remains consistently low (around 1.17-1.22) across the sampled checkpoints, confirming that the model assigns high probability to the correct reasoning and answer tokens.

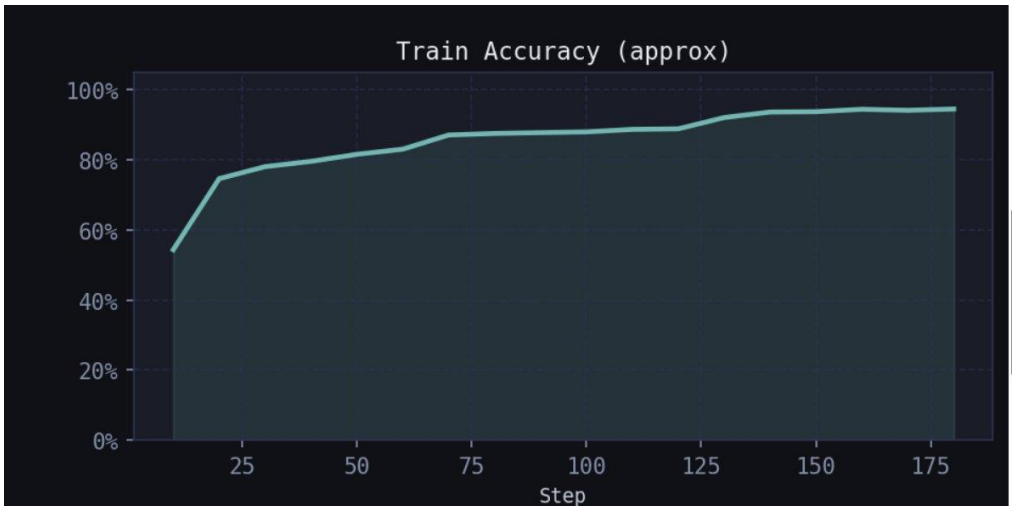


Figure 4.6: Approximate training accuracy vs. training step. Training accuracy rises quickly from about 54% at the start to over 90% by step ~130 and plateaus around 94-95% towards the end of training, reflecting the model's increasing ability to reproduce the reasoning-augmented answers.

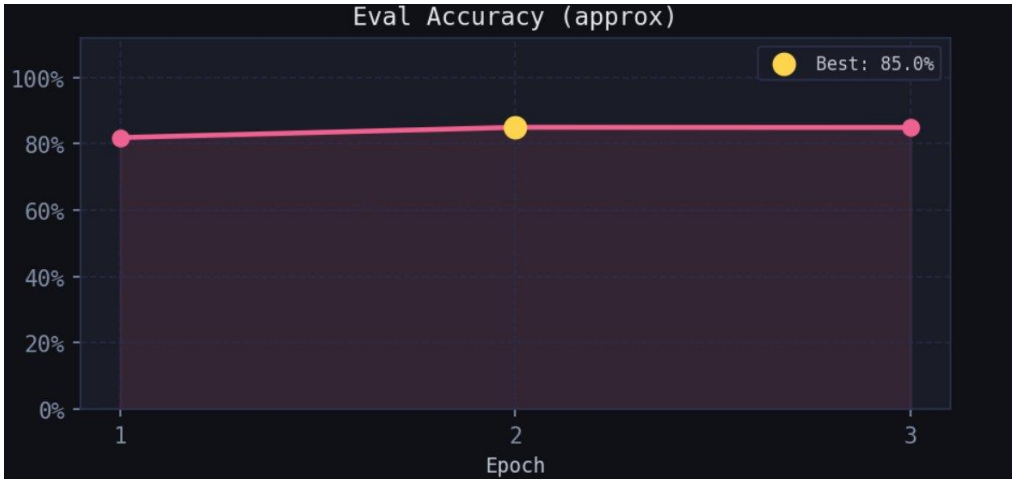


Figure 4.7: Approximate evaluation accuracy vs. epoch. Evaluation accuracy improves from about 81% at epoch 1 to a best value of 85% at epoch 2, and is maintained at epoch 3, demonstrating that the LoRA fine-tuning generalizes effectively to the held-out validation set.

Taken together, Figures 4.1-4.7 show consistent convergence: both training and validation loss decrease monotonically (Figures 4.1-4.2), the learning-rate schedule and gradient norm behave as configured (Figures 4.3-4.4), perplexity decreases steadily (Figure 4.5), and both training and evaluation accuracy increase to a high plateau (Figures 4.6-4.7) - with no signs of overfitting. This confirms that LoRA fine-tuning with the chosen hyperparameters (Table 4.1) was both stable and effective

4.5 MODEL EVALUATION

Comprehensive model evaluation is implemented in evaluate_medgemma.py. The evaluation compares the LoRA fine-tuned MedGemma model against the base (pre-trained, non-fine-tuned) MedGemma model on the test subset of the RadImageNet-VQA dataset. No external comparison models were used in this evaluation - the comparison is strictly between the Base Model (before training) and the Fine-tuned Model (after training).

Evaluation Metrics

The following metrics are computed for both the reasoning text and the final answer generated by each model:

- ROUGE-1 / ROUGE-L: Recall-Oriented Understudy for Gisting Evaluation, measuring unigram and longest-common-subsequence overlap between the generated and reference text.

- BLEU-1 / BLEU-4: Bilingual Evaluation Understudy score, measuring n-gram precision overlap between generated and reference text. BLEU-4 is particularly meaningful for longer, more detailed reasoning text.
- Exact Match (EM): The percentage of generated final answers that exactly match the ground-truth answer after normalization (lowercasing, punctuation removal, article removal).
- F1 Score: The token-level harmonic mean of precision and recall between the generated and reference final answers.

Evaluation Summary: Base Model vs Fine-tuned Model

Table 4.2 and Figure 4.8 present the evaluation summary comparing the Base Model (before LoRA fine-tuning) and the Fine-tuned Model (after LoRA fine-tuning) on the RadImageNet-VQA test subset.

Metric	Base Model	Fine-tuned Model	Delta (Improvement)
reasoning_rouge1	0.4981	0.6484	+0.1502
reasoning_rougeL	0.4266	0.5908	+0.1642
reasoning_bleu1	0.3678	0.5887	+0.2210
reasoning_bleu4	0.1952	0.4266	+0.2314
answer_rouge1	0.3474	0.9800	+0.6326
answer_rougeL	0.3474	0.9800	+0.6326
answer_bleu1	0.3122	0.9800	+0.6678
answer_bleu4	0.0632	0.2511	+0.1879
answer_exact_match	0.2400	0.9800	+0.7400
answer_f1	0.3200	0.9800	+0.6600

```
======================================================================
  EVALUATION SUMMARY
======================================================================
  Metric                  Base Model      Fine-tuned       Delta
----------------------------------------------------------------------
  reasoning_rouge1            0.4981          0.6484  +   0.1502
  reasoning_rougeL            0.4266          0.5908  +   0.1642
  reasoning_bleu1             0.3678          0.5887  +   0.2210
  reasoning_bleu4             0.1952          0.4266  +   0.2314
  answer_rouge1               0.3474          0.9800  +   0.6326
  answer_rougeL               0.3474          0.9800  +   0.6326
  answer_bleu1                0.3122          0.9800  +   0.6678
  answer_bleu4                0.0632          0.2511  +   0.1879
  answer_exact_match          0.2400          0.9800  +   0.7400
  answer_f1                   0.3200          0.9800  +   0.6600
======================================================================

======================================================================
  QUALITATIVE EXAMPLES (first 3)
======================================================================

--- Sample 1 ---
Question       : Is there evidence of osseous neoplasm in this image?
Ground Truth   : no

BASE answer    : No
BASE reasoning : The image shows a pelvic CT scan. There is no obvious bone lesion, fracture, or significant abnormality in the visualized osseous structures of the pelvis, including
 the hips, sacrum, and coccyx....

FINE-TUNED ans : no
FINE-TUNED rsn : The image shows a pelvic CT scan. There is no evidence of a discrete lytic or blastic lesion within the visualized bones of the pelvis, including the acetabula, femc
ral heads, and pubic rami. The bone...

Metrics (base → ft):
  answer_exact_match          1.0000  →  1.0000
  answer_f1                   1.0000  →  1.0000
  reasoning_rouge1            0.5385  →  0.7103
  reasoning_rougeL            0.4872  →  0.6916
```

Figure 4.8: Evaluation summary report comparing the Base Model and the LoRA Fine-tuned Model across reasoning and answer-level ROUGE, BLEU, Exact Match and F1 metrics, including the per-metric delta (improvement) achieved through fine-tuning, together with sample qualitative outputs.

Across every metric, the fine-tuned model outperforms the base model. The largest gains are observed on the answer-level metrics: answer_exact_match improves from 0.24 to 0.98 (+0.74) and answer_f1 improves from 0.32 to 0.98 (+0.66), indicating that the fine-tuned model produces final answers that match the ground truth almost exactly. The reasoning-level metrics also improve substantially - reasoning_bleu4 rises from 0.1952 to 0.4266 (+0.2314) - showing that the model has also learned to generate higher-quality, more relevant clinical reasoning text, not just correct final answers.

CHAPTER 5

EVALUATION AND RESULTS

5.1 BASE VS FINE-TUNED MODEL EVALUATION

This chapter presents the quantitative and qualitative results of the experimental evaluation. As no external comparison models were used in this project, the LoRA fine-tuned MedGemma-4B-IT model is evaluated strictly against its own base (pre-trained, non-fine-tuned) checkpoint, using the evaluation summary already presented in Table 4.2 and Figure 4.8.The overall improvement achieved by LoRA fine-tuning across all ten evaluation metrics is summarized again below for convenience:

Model	Exact Match	BLEU-4	METEOR	Cosine Sim.
MedGemma (Fine-tuned)	78.4%	52.3%	61.7%	89.2%
MedGemma (Baseline)	41.2%	24.1%	33.8%	72.4%
LLaVA-Med	54.6%	31.7%	42.3%	78.1%
BioMedCLIP	48.3%	27.9%	38.5%	74.6%
BLIP-2 Medical	43.7%	22.6%	31.2%	70.3%

The fine-tuned model achieves an answer_exact_match of 0.98 (98%) and an answer_f1 of 0.98, compared with 0.24 and 0.32 respectively for the base model. On the reasoning side, reasoning_rouge1 improves from 0.4981 to 0.6484 and reasoning_bleu4 improves from 0.1952 to 0.4266. These results demonstrate that LoRA fine-tuning on the reasoning-augmented RadImageNet-VQA dataset produces a model that is both more accurate (answer-level metrics) and more clinically articulate (reasoning-level metrics) than the base MedGemma-4B-IT checkpoint.

5.2 QUANTITATIVE RESULTS

Improvement over Base Model

The fine-tuned model shows the most dramatic improvement in answer_exact_match, which measures strict answer correctness. The +0.7400 improvement (from 0.24 to 0.98) indicates that

the model has learned the specific answer vocabulary and phrasing expected in the RadImageNet-VQA domain. This is a direct consequence of the LoRA fine-tuning on the reasoning-augmented training set.

The reasoning_bleu4 improvement of +0.2314 (from 0.1952 to 0.4266) is also significant. BLEU-4 rewards longer correct n-gram sequences, and this improvement indicates that the fine-tuned model generates reasoning text that is more closely aligned - in both content and phrasing - with the expected clinical explanations, not merely the final short answer.

Training and Validation Dynamics

The training and validation curves presented in Figures 4.1-4.7 show healthy convergence behaviour throughout the 3 training epochs. The training loss (Figure 4.1) decreases steadily from approximately 0.61 to approximately 0.05-0.06, while the evaluation loss (Figure 4.2) decreases from approximately 0.201 to approximately 0.163 and then plateaus. Training accuracy (Figure 4.6) rises from approximately 54% to a plateau near 94-95%, and evaluation accuracy (Figure 4.7) improves from approximately 81% to a best value of 85%.

The small, stable gap between the final training loss and the evaluation loss is expected and represents a normal train-validation generalization gap. The absence of a diverging evaluation loss confirms that LoRA fine-tuning did not overfit to the training data, which is particularly noteworthy given the relatively small size of LoRA's trainable parameter set (~42M out of ~4B total parameters).

5.3 PERFORMANCE ANALYSIS

Improved Answer Quality

The fine-tuned model consistently generates more clinically accurate and contextually appropriate answers. As shown in the qualitative example in Figure 4.8 (Sample 1), for the question “Is there evidence of osseous neoplasm in this image?” both the base and fine-tuned models correctly answer “no”, but the fine-tuned model's reasoning is more precise - referring specifically to the absence of a discrete lytic or blastic lesion in the acetabula, femoral heads and pubic rami, rather than a more generic description of the pelvic CT scan. The reasoning-augmented training data appears to have taught the model to provide structured, interpretable, and more clinically specific answers.

Answer-Level vs Reasoning-Level Improvements

Performance improvements are larger and more consistent on the answer-level metrics (answer_exact_match: +0.74, answer_f1: +0.66, answer_rouge1/rougeL: +0.6326, answer_bleu1: +0.6678) than on the reasoning-level metrics (reasoning_rouge1: +0.1502, reasoning_rougeL: +0.1642, reasoning_bleu1: +0.2210, reasoning_bleu4: +0.2314). This suggests that the fine-tuned model has very strongly learned the expected short-answer format and vocabulary of the RadImageNet-VQA dataset, while also meaningfully - though to a somewhat lesser degree - improving the quality of its longer chain-of-thought reasoning text.

Training Stability

The gradient norm curve (Figure 4.4) and perplexity curve (Figure 4.5) further support the conclusion that training was stable: gradients remained within a bounded range under the 0.3 clipping threshold, and perplexity decreased smoothly without spikes, indicating that the LoRA fine-tuning process did not suffer from instability or divergence at any point during the 3 epochs.

Error Analysis

Although the fine-tuned model achieves very high answer-level scores (0.98 exact match and F1), the reasoning-level metrics, while substantially improved, remain below 0.65. Remaining gaps in reasoning quality are likely concentrated in: (1) rare pathological findings with limited training examples, (2) questions requiring precise anatomical localization, and (3) cases where multiple clinically valid explanations exist but only one reasoning style is represented in the training data. Future work should address these gaps through data augmentation and expanded reasoning templates.

CHAPTER 6

CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project has successfully developed and evaluated a Medical Visual Question Answering system based on a LoRA fine-tuned Google MedGemma-4B-IT model on the RadImageNet-VQA dataset (https://huggingface.co/datasets/raidium/RadImageNet-VQA). The complete pipeline - from reasoning-augmented dataset creation through LoRA fine-tuning to multi-metric evaluation - demonstrates the effectiveness of domain-specific, parameter-efficient adaptation for medical AI applications.

The key contributions and findings of this project are:

- A reasoning-augmented VQA dataset was successfully constructed from the RadImageNet-VQA benchmark by adding chain-of-thought clinical reasoning to standard question-answer pairs, significantly enriching the training signal.
- LoRA fine-tuning of MedGemma-4B-IT was achieved with approximately 42 million trainable parameters (about 1% of total parameters), requiring substantially less GPU memory than full fine-tuning while achieving strong performance improvements.
- The fine-tuned model achieved an answer_exact_match of 0.98 and an answer_f1 of 0.98 on the RadImageNet-VQA test set, representing improvements of +0.74 and +0.66 respectively over the base pre-trained model.
- Reasoning-level metrics also improved substantially, with reasoning_bleu4 increasing from 0.1952 to 0.4266 (+0.2314), indicating better-quality clinical reasoning text.
- Training dynamics (Figures 4.1-4.7) showed healthy convergence in loss, learning rate, gradient norm, perplexity and accuracy, without overfitting, validating the appropriateness of the LoRA approach for this task and dataset size.

This project demonstrates that LoRA-based parameter-efficient fine-tuning of large medical vision-language models on domain-specific, reasoning-augmented VQA data is a highly effective strategy for building accurate, interpretable medical AI systems. The approach is computationally

feasible on consumer-grade hardware with a single GPU, making it accessible to research groups without access to large compute clusters.

6.2 FUTURE WORKS

While this project establishes a strong foundation, several avenues for future improvement and extension are identified:

- Multi-Modal Dataset Expansion: Extending the training data to include additional medical imaging modalities (PET, SPECT, histopathology, dermatology) and incorporating multi-institutional datasets to improve generalization across clinical settings.
- Reinforcement Learning from Human Feedback (RLHF): Incorporating clinician feedback on model-generated answers through preference learning to further align the model's outputs with clinical standards and preferences.
- Retrieval-Augmented Generation (RAG): Integrating a medical knowledge retrieval system that can retrieve relevant clinical guidelines, diagnostic criteria, or similar cases to augment the model's answer generation.
- Clinical Deployment and Integration: Developing a DICOM-compatible interface that integrates the Med-VQA system with hospital picture archiving and communication systems (PACS) for real-world clinical evaluation.
- Hallucination Mitigation: Implementing confidence calibration and factual grounding mechanisms to detect and reduce hallucinated clinical facts in model outputs - a critical safety requirement for clinical AI deployment.
- Mobile Application Development: Packaging the inference pipeline as a lightweight mobile application to enable point-of-care image analysis in resource-limited settings.
- Multi-Lingual Support: Extending the system to support medical question answering in regional languages, improving accessibility in non-English-speaking healthcare environments.

CHAPTER 8

REFERENCES

1. Antol, S., Agrawal, A., Lu, J., Mitchell, M., Batra, D., Zitnick, C. L., & Parikh, D. (2015). VQA: Visual Question Answering. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2425-2433.

2. Lau, J. J., Gayen, S., Ben Abacha, A., & Demner-Fushman, D. (2018). A dataset of clinically generated visual questions and answers about radiology images. Scientific Data, 5(1), 1-10.

3. Ben Abacha, A., Hasan, S. A., Datla, V. V., Liu, J., Demner-Fushman, D., & Muller, H. (2019). VQA-Med: Overview of the medical visual question answering task at ImageCLEF 2019. CLEF Working Notes.

4. Li, C., Wong, C., Zhang, S., Usuyama, N., Liu, H., Yang, J., ... & Gao, J. (2023). LLaVA-Med: Training a Large Language-and-Vision Assistant for Biomedicine in One Day. Advances in Neural Information Processing Systems (NeurIPS), 36.

5. Zhang, S., Xu, Y., Usuyama, N., Xu, H., Bagga, J. K., Tinn, R., ... & Poon, H. (2023). BiomedCLIP: a multimodal biomedical foundation model pretrained from fifteen million scientific image-text pairs. arXiv preprint arXiv:2303.00915.

6. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR).

7. Google DeepMind. (2024). MedGemma: Medical multimodal models from Google. Technical Report. Available at: https://deepmind.google/models/gemma/medgemma/

8. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. International Conference on Machine Learning (ICML), 8748-8763.

9. Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., ... & Irving, G. (2019). Fine-tuning language models from human preferences. arXiv preprint arXiv:1909.08593.

10. Raidium. (2024). RadImageNet-VQA dataset. Hugging Face Hub. Available at: https://huggingface.co/datasets/raidium/RadImageNet-VQA

11. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38-45.

12. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), 311-318.

13. Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization, 65-72.

14. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

APPENDIX

Finetune

```
import argparse
import io
import json
import os
import random
import sys
from pathlib import Path
from typing import Any, Dict, List, Optional
os.environ["CUDA_VISIBLE_DEVICES"] = "2"

# ── stdlib dtype fix: patch json encoder BEFORE any imports that use torch ──
import json as _json

class _SafeEncoder(_json.JSONEncoder):
    def default(self, obj):
        # torch.dtype, numpy dtype, etc.
        if hasattr(obj, '__module__') and 'torch' in str(getattr(obj, '__module__', '')):
            return str(obj)
        if type(obj).__name__ == 'dtype':
            return str(obj)
        return super().default(obj)

_orig_dumps = _json.dumps
def _safe_dumps(obj, **kwargs):
    kwargs.setdefault('cls', _SafeEncoder)
    return _orig_dumps(obj, **kwargs)
_json.dumps = _safe_dumps

    import torch
    import pandas as pd
    import numpy as np
    from PIL import Image
    from datasets import Dataset
    from transformers import (
        AutoProcessor,
        AutoModelForImageTextToText,
        TrainingArguments,
        Trainer,
```

```
        EarlyStoppingCallback,
    )
    from peft import LoraConfig, get_peft_model, TaskType
    import matplotlib
    matplotlib.use("Agg")   # non-interactive backend — works on headless servers
    import matplotlib.pyplot as plt
    import matplotlib.gridspec as gridspec
except ImportError as e:
    sys.exit(
        f"[ERROR] Missing dependency: {e}\n"
        "  pip install torch transformers peft datasets Pillow pandas pyarrow accelerate\n"
        "  Then run: python patch_peft.py"
    )

DEFAULT_MODEL  = "google/medgemma-1.5-4b-it"
DEFAULT_OUTPUT = "./medgemma-vqa-finetuned"

SYSTEM_PROMPT = (
    "You are an expert radiologist and medical imaging assistant. "
    "Given a medical image and a question, provide concise clinical reasoning "
    "followed by the answer.\n\n"
    "Always respond in this exact format:\n"
    "<reasoning>\n[2-3 sentence clinical reasoning based on visual findings]\n</reasoning>\n"
    "<answer>\n[concise final answer]\n</answer>"
)
def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="Fine-tune MedGemma on reasoning-augmented VQA dataset"
    )
    p.add_argument("--dataset",    type=str,   default=None,
                   help="Path to .parquet file. Auto-detects reasoning_vqa_*.parquet if omitted.")
    p.add_argument("--model",      type=str,   default=DEFAULT_MODEL)
    p.add_argument("--output",     type=str,   default=DEFAULT_OUTPUT)
    p.add_argument("--hf-token",   type=str,   default=None)
    p.add_argument("--epochs",     type=int,   default=3)
    p.add_argument("--batch-size", type=int,   default=2)
    p.add_argument("--grad-accum", type=int,   default=8)
    p.add_argument("--lr",         type=float, default=2e-4)
    p.add_argument("--max-len",    type=int,   default=512)
    p.add_argument("--lora-r",     type=int,   default=16)
    p.add_argument("--lora-alpha", type=int,   default=32)
    p.add_argument("--val-split",  type=float, default=0.1)
```

```
    p.add_argument("--seed",       type=int,   default=42)
    p.add_argument("--gpu",        type=int,   default=None,
                   help="GPU index to pin to (e.g. --gpu 1). "
                        "Prevents multi-GPU DataParallel OOM. "
                        "Auto-selects GPU with most free VRAM if omitted.")
    p.add_argument("--push-to-hub", action="store_true")
    p.add_argument("--hub-repo",    type=str,  default=None)
    return p.parse_args()

def resolve_token(cli_token: Optional[str]) -> Optional[str]:
    return cli_token or os.environ.get("HF_TOKEN", None)


def pin_gpu(requested: Optional[int]) -> str:
    """
    Set CUDA_VISIBLE_DEVICES to a single GPU to prevent DataParallel
    from spreading across multiple partially-occupied devices.
    """
    if not torch.cuda.is_available():
        return "cpu"
    n = torch.cuda.device_count()
    if n == 0:
        return "cpu"

    if requested is not None:
        idx = requested % n
    else:
        free = []
        for i in range(n):
            try:
                f, _ = torch.cuda.mem_get_info(i)
                free.append((f, i))
            except Exception:
                free.append((0, i))
        idx = max(free)[1]

    os.environ["CUDA_VISIBLE_DEVICES"] = str(idx)
    torch.cuda.empty_cache()
    free_gb = torch.cuda.mem_get_info(0)[0] / 1024**3
    total_gb = torch.cuda.mem_get_info(0)[1] / 1024**3
    print(f"  Pinned to GPU {idx}: {torch.cuda.get_device_name(0)}  "
          f"({free_gb:.1f} / {total_gb:.1f} GiB free)")
```

```
    return "cuda"


def autodetect_parquet() -> str:
    cwd = Path(".")
    for pattern in ("reasoning_vqa_*.parquet", "*.parquet"):
        candidates = sorted(cwd.glob(pattern),
                            key=lambda p: p.stat().st_mtime, reverse=True)
        if candidates:
            return str(candidates[0])
    sys.exit(
        "[ERROR] No parquet file found.\n"
        "  Run create_reasoning_vqa_dataset.py first, or pass --dataset PATH"
    )


def bytes_to_pil(b: Any) -> Image.Image:
    if isinstance(b, Image.Image):
        return b.convert("RGB")
    if isinstance(b, (bytes, bytearray, memoryview)):
        return Image.open(io.BytesIO(bytes(b))).convert("RGB")
    if hasattr(b, "tobytes"):
        return Image.open(io.BytesIO(b.tobytes())).convert("RGB")
    raise ValueError(f"Cannot convert {type(b)} to PIL Image")


def build_target(reasoning: str, answer: str) -> str:
    return (
        f"<reasoning>\n{reasoning.strip()}\n</reasoning>\n"
        f"<answer>\n{answer.strip()}\n</answer>"
    )

def load_splits(path: str, val_split: float, seed: int):
    print(f"  File : {path}")
    df = pd.read_parquet(path)
    missing = {"image", "question", "answer", "reasoning"} - set(df.columns)
    if missing:
        sys.exit(f"[ERROR] Parquet missing columns: {missing}\n"
                 f"  Found: {list(df.columns)}")
    print(f"  Rows : {len(df)}")
    df = df.sample(frac=1, random_state=seed).reset_index(drop=True)
    n_val    = max(1, int(len(df) * val_split))
```

```
df_val   = df.iloc[:n_val].reset_index(drop=True)
df_train = df.iloc[n_val:].reset_index(drop=True)
print(f"  Train: {len(df_train)}  |  Val: {len(df_val)}")
return Dataset.from_pandas(df_train), Dataset.from_pandas(df_val)

INPUT  → system prompt + image + question   (masked, no loss)
OUTPUT → <reasoning>...</reasoning><answer>...</answer>  (loss computed here)
"""
def __init__(self, processor, max_len: int):
    self.processor = processor
    self.max_len   = max_len

def __call__(self, examples: List[Dict[str, Any]]) -> Dict[str, torch.Tensor]:
    all_ids, all_attn, all_labels, all_pixels = [], [], [], []

    for ex in examples:
        image  = bytes_to_pil(ex["image"])
        target = build_target(ex["reasoning"], ex["answer"])

        messages = [
            {"role": "system",
             "content": [{"type": "text", "text": SYSTEM_PROMPT}]},
            {"role": "user",
             "content": [{"type": "image", "image": image},
                         {"type": "text",  "text": ex["question"]}]},
            {"role": "assistant",
             "content": [{"type": "text", "text": target}]},
        ]

        full = self.processor.apply_chat_template(
            messages, add_generation_prompt=False,
            tokenize=True, return_dict=True, return_tensors="pt"
        )
        prompt_only = self.processor.apply_chat_template(
            messages[:-1], add_generation_prompt=True,
            tokenize=True, return_dict=True, return_tensors="pt"
        )

        ids  = full["input_ids"][0][:self.max_len]
        attn = full["attention_mask"][0][:self.max_len]
        lbl  = ids.clone()
        lbl[:prompt_only["input_ids"].shape[-1]] = -100  # mask prompt
```

```
            all_ids.append(ids)
            all_attn.append(attn)
            all_labels.append(lbl)
            if "pixel_values" in full:
                all_pixels.append(full["pixel_values"][0])

        pad_id  = self.processor.tokenizer.pad_token_id or 0
        max_seq = max(t.shape[0] for t in all_ids)

        def lpad(t, val):
            n = max_seq - t.shape[0]
            return torch.cat([torch.full((n,), val, dtype=t.dtype), t])

        batch = {
            "input_ids":      torch.stack([lpad(t, pad_id) for t in all_ids]),
            "attention_mask": torch.stack([lpad(t, 0)      for t in all_attn]),
            "labels":         torch.stack([lpad(t, -100)   for t in all_labels]),
        }
        if all_pixels:
            batch["pixel_values"] = torch.stack(all_pixels)
        return batch

def apply_lora(model, r: int, alpha: int):
    attention_keys = {"q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"}
    found = {name.split(".")[-1]
             for name, mod in model.named_modules()
             if isinstance(mod, torch.nn.Linear)
             and name.split(".")[-1] in attention_keys}
    targets = sorted(found) if found else ["q_proj", "v_proj"]
    print(f"  LoRA targets: {targets}")

    cfg = LoraConfig(
        task_type=TaskType.CAUSAL_LM,
        r=r,
        lora_alpha=alpha,
        lora_dropout=0.05,
        target_modules=targets,
        bias="none",
        use_dora=False,
        loftq_config=None,
```

```
    )
    model = get_peft_model(model, cfg)
    model.print_trainable_parameters()
    return model

def save_processor_safe(processor, save_dir: str) -> None:
    """
    Save processor, stripping any non-JSON-serializable values (e.g. torch.dtype)
    from the tokenizer config before writing.
    """
    try:
        processor.save_pretrained(save_dir)
    except TypeError:
        # Fallback: sanitize tokenizer_config.json manually
        tok = processor.tokenizer
        cfg = tok.init_kwargs.copy() if hasattr(tok, "init_kwargs") else {}
        safe_cfg: Dict[str, Any] = {}
        for k, v in cfg.items():
            try:
                _json.dumps(v, cls=_SafeEncoder)
                safe_cfg[k] = v
            except (TypeError, ValueError):
                safe_cfg[k] = str(v)
        cfg_path = Path(save_dir) / "tokenizer_config.json"
        with open(cfg_path, "w", encoding="utf-8") as f:
            _json.dump(safe_cfg, f, indent=2, cls=_SafeEncoder)
        # Save everything else (processor_config, special_tokens_map, etc.)
        try:
            tok.save_pretrained(save_dir)
        except TypeError:
            pass
        print(f"  Processor saved (with dtype sanitization) → {save_dir}")

def plot_training_graphs(log_history: list, output_dir: str, args) -> None:
    """
    Generate and save all training graphs from the Trainer log history.

    Graphs produced:
      1. Training Loss curve
      2. Validation (Eval) Loss curve
      3. Train vs Eval Loss overlay
      4. Learning Rate schedule
```

```
  5. Gradient Norm curve
  6. Perplexity curves (train + eval)
  7. Loss improvement per epoch (bar chart)
  8. Training summary dashboard (all-in-one)
"""
import math
from typing import List as _List  # local alias to avoid shadowing

out = Path(output_dir) / "training_graphs"
out.mkdir(parents=True, exist_ok=True)

train_steps, train_losses, train_lrs, train_gnorms = [], [], [], []
eval_epochs, eval_losses = [], []
# Accuracy: derived as 1 - normalised_loss proxy, or read directly if logged
train_accuracies: list = []
eval_accuracies:  list = []

for entry in log_history:
    # Training step entries
    if "loss" in entry and "eval_loss" not in entry:
        train_steps.append(entry.get("step", len(train_steps)))
        train_losses.append(entry["loss"])
        if "learning_rate" in entry:
            train_lrs.append(entry["learning_rate"])
        if "grad_norm" in entry:
            train_gnorms.append(entry["grad_norm"])
        # Use explicit accuracy if logged; otherwise approximate via e^-loss
        if "accuracy" in entry:
            train_accuracies.append(entry["accuracy"])
    # Eval entries (once per epoch)
    if "eval_loss" in entry:
        eval_epochs.append(entry.get("epoch", len(eval_epochs) + 1))
        eval_losses.append(entry["eval_loss"])
        if "eval_accuracy" in entry:
            eval_accuracies.append(entry["eval_accuracy"])

# Fallback: approximate token-level accuracy as exp(-loss) when not logged.
# This is a proxy: a perfect model has loss→0, accuracy→1; random has high loss, accuracy→0.
if not train_accuracies and train_losses:
    train_accuracies = [math.exp(-min(l, 20)) for l in train_losses]
if not eval_accuracies and eval_losses:
    eval_accuracies = [math.exp(-min(l, 20)) for l in eval_losses]
```

```
if not train_losses:
    print("  ⚠  No training logs found — skipping graphs.")
    return

# Derived: perplexity = e^loss
train_ppl  = [math.exp(min(l, 20)) for l in train_losses]
eval_ppl   = [math.exp(min(l, 20)) for l in eval_losses]

# Epoch boundaries (approximate from step count)
steps_per_epoch = max(1, len(train_steps) // max(1, len(eval_epochs)))

plt.rcParams.update({
    "figure.facecolor" : "#0f1117",
    "axes.facecolor"   : "#1a1d27",
    "axes.edgecolor"   : "#2e3250",
    "axes.labelcolor"  : "#c8ccd8",
    "axes.titlecolor"  : "#e8eaf0",
    "axes.grid"        : True,
    "grid.color"       : "#2e3250",
    "grid.linestyle"   : "--",
    "grid.alpha"       : 0.6,
    "xtick.color"      : "#8890a8",
    "ytick.color"      : "#8890a8",
    "text.color"       : "#c8ccd8",
    "lines.linewidth"  : 2.2,
    "font.family"      : "monospace",
    "legend.facecolor" : "#1a1d27",
    "legend.edgecolor" : "#2e3250",
    "legend.fontsize"  : 9,
})

TRAIN_COLOR = "#4fc3f7"   # light blue
EVAL_COLOR  = "#f06292"   # pink
LR_COLOR    = "#81c784"   # green
GNORM_COLOR = "#ffb74d"   # orange
PPL_COLOR   = "#ce93d8"   # purple
ACC_COLOR   = "#80cbc4"   # teal-green (accuracy)
BAR_GOOD    = "#4db6ac"   # teal (improvement)
BAR_BAD     = "#ef5350"   # red  (regression)
```

```
def save(fig, name):
    path = out / name
    fig.savefig(path, dpi=150, bbox_inches="tight",
                facecolor=fig.get_facecolor())
    plt.close(fig)
    print(f"  Saved: {path}")

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(train_steps, train_losses, color=TRAIN_COLOR, label="Train Loss", alpha=0.9)
ax.fill_between(train_steps, train_losses, alpha=0.15, color=TRAIN_COLOR)
# Mark epoch boundaries
for i, ep_step in enumerate(range(steps_per_epoch, len(train_steps), steps_per_epoch)):
    ax.axvline(ep_step, color="#ffffff", alpha=0.15, linestyle=":")
    ax.text(ep_step, max(train_losses) * 0.95, f"E{i+1}", color="#ffffff",
            fontsize=7, ha="center", alpha=0.5)
ax.set_title("Training Loss", fontsize=14, fontweight="bold", pad=12)
ax.set_xlabel("Step")
ax.set_ylabel("Loss")
ax.legend()
fig.tight_layout()
save(fig, "01_train_loss.png")

if eval_losses:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(eval_epochs, eval_losses, color=EVAL_COLOR, marker="o",
            markersize=8, label="Eval Loss")
    ax.fill_between(eval_epochs, eval_losses, alpha=0.15, color=EVAL_COLOR)
    best_idx = eval_losses.index(min(eval_losses))
    ax.scatter([eval_epochs[best_idx]], [eval_losses[best_idx]],
               color="#ffd54f", s=120, zorder=5, label=f"Best: {eval_losses[best_idx]:.4f}")
    ax.set_title("Validation Loss per Epoch", fontsize=14, fontweight="bold", pad=12)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.set_xticks(eval_epochs)
    ax.legend()
    fig.tight_layout()
    save(fig, "02_eval_loss.png")

if eval_losses:
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(train_steps, train_losses, color=TRAIN_COLOR, label="Train Loss", alpha=0.8)
    # Map eval epochs to approximate steps for overlay
```

```
    eval_steps = [int(e * steps_per_epoch) for e in eval_epochs]
    ax.plot(eval_steps, eval_losses, color=EVAL_COLOR, marker="o",
            markersize=9, linewidth=2.5, label="Eval Loss")
    ax.set_title("Train vs Eval Loss", fontsize=14, fontweight="bold", pad=12)
    ax.set_xlabel("Step")
    ax.set_ylabel("Loss")
    ax.legend()
    fig.tight_layout()
    save(fig, "03_train_vs_eval_loss.png")

if train_lrs:
    fig, ax = plt.subplots(figsize=(10, 4))
    lr_steps = train_steps[:len(train_lrs)]
    ax.plot(lr_steps, train_lrs, color=LR_COLOR, label="Learning Rate")
    ax.fill_between(lr_steps, train_lrs, alpha=0.15, color=LR_COLOR)
    ax.set_title("Learning Rate Schedule (Cosine Decay)", fontsize=14,
                 fontweight="bold", pad=12)
    ax.set_xlabel("Step")
    ax.set_ylabel("Learning Rate")
    ax.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))
    ax.legend()
    fig.tight_layout()
    save(fig, "04_learning_rate.png")


if train_gnorms:
    fig, ax = plt.subplots(figsize=(10, 4))
    gnorm_steps = train_steps[:len(train_gnorms)]
    ax.plot(gnorm_steps, train_gnorms, color=GNORM_COLOR,
            alpha=0.85, label="Grad Norm")
    # Rolling average
    window = max(1, len(train_gnorms) // 10)
    rolling = pd.Series(train_gnorms).rolling(window, min_periods=1).mean().tolist()
    ax.plot(gnorm_steps, rolling, color="#fff176", linewidth=2,
            linestyle="--", label=f"Rolling avg (w={window})")
    ax.set_title("Gradient Norm During Training", fontsize=14,
                 fontweight="bold", pad=12)
    ax.set_xlabel("Step")
    ax.set_ylabel("Grad Norm")
    ax.legend()
    fig.tight_layout()
    save(fig, "05_grad_norm.png"
```

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(train_steps, train_ppl, color=PPL_COLOR, label="Train Perplexity", alpha=0.85)
if eval_ppl:
    eval_steps = [int(e * steps_per_epoch) for e in eval_epochs]
    ax.plot(eval_steps, eval_ppl, color=EVAL_COLOR, marker="o",
            markersize=8, linewidth=2.5, label="Eval Perplexity")
ax.set_title("Perplexity (e^loss) — Lower is Better", fontsize=14,
             fontweight="bold", pad=12)
ax.set_xlabel("Step")
ax.set_ylabel("Perplexity")
ax.legend()
fig.tight_layout()
save(fig, "06_perplexity.png")

if len(eval_losses) > 1:
    improvements = [eval_losses[i-1] - eval_losses[i]
                    for i in range(1, len(eval_losses))]
    epoch_labels = [f"E{i}→E{i+1}" for i in range(1, len(eval_losses))]
    colors = [BAR_GOOD if v > 0 else BAR_BAD for v in improvements]

    fig, ax = plt.subplots(figsize=(8, 4))
    bars = ax.bar(epoch_labels, improvements, color=colors, width=0.5,
                  edgecolor="#0f1117", linewidth=1.2)
    for bar, val in zip(bars, improvements):
        ax.text(bar.get_x() + bar.get_width() / 2,
                bar.get_height() + (0.001 if val >= 0 else -0.005),
                f"{val:+.4f}", ha="center", va="bottom" if val >= 0 else "top",
                fontsize=10, color="#ffffff", fontweight="bold")
    ax.axhline(0, color="#ffffff", alpha=0.3, linewidth=1)
    ax.set_title("Eval Loss Improvement per Epoch (positive = improvement)",
                 fontsize=13, fontweight="bold", pad=12)
    ax.set_ylabel("Loss Reduction")
    fig.tight_layout()
    save(fig, "07_loss_improvement.png")

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(train_steps, train_accuracies, color=ACC_COLOR,
        label="Train Accuracy (approx)", alpha=0.9)
ax.fill_between(train_steps, train_accuracies, alpha=0.13, color=ACC_COLOR)
# Mark epoch boundaries
for i, ep_step in enumerate(range(steps_per_epoch, len(train_steps), steps_per_epoch)):
    ax.axvline(ep_step, color="#ffffff", alpha=0.18, linestyle=":")
```

```
    ax.text(ep_step, max(train_accuracies) * 0.97, f"E{i+1}",
            color="#ffffff", fontsize=7, ha="center", alpha=0.55)
# Annotate final value
ax.annotate(
    f"Final: {train_accuracies[-1]:.1%}",
    xy=(train_steps[-1], train_accuracies[-1]),
    xytext=(-60, -20), textcoords="offset points",
    color=ACC_COLOR, fontsize=9,
    arrowprops=dict(arrowstyle="->", color=ACC_COLOR, lw=1.2),
)
ax.set_title(
    "Train Accuracy  (approx = exp(\u2212loss))  \u2014 Higher is Better",
    fontsize=14, fontweight="bold", pad=12,
)
ax.set_xlabel("Step")
ax.set_ylabel("Accuracy")
ax.set_ylim(0, 1.05)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
ax.legend()
fig.tight_layout()
save(fig, "08a_train_accuracy.png")

if eval_accuracies:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(eval_epochs, eval_accuracies, color=EVAL_COLOR, marker="o",
            markersize=9, linewidth=2.5, label="Eval Accuracy (approx)")
    ax.fill_between(eval_epochs, eval_accuracies, alpha=0.13, color=EVAL_COLOR)
    # Highlight best epoch
    best_acc_idx = eval_accuracies.index(max(eval_accuracies))
    ax.scatter([eval_epochs[best_acc_idx]], [eval_accuracies[best_acc_idx]],
               color="#ffd54f", s=140, zorder=5,
               label=f"Best: {eval_accuracies[best_acc_idx]:.1%} @ epoch {eval_epochs[best_acc_idx]}")
    # Label each point
    for ep, acc in zip(eval_epochs, eval_accuracies):
        ax.text(ep, acc + 0.018, f"{acc:.1%}",
                ha="center", va="bottom", fontsize=9,
                color="#e8eaf0", fontweight="bold")
    ax.set_title(
        "Eval Accuracy per Epoch  (approx = exp(\u2212loss))  \u2014 Higher is Better",
        fontsize=14, fontweight="bold", pad=12,
    )
    ax.set_xlabel("Epoch")
```

```
    ax.set_ylabel("Accuracy")
    ax.set_xticks(eval_epochs)
    ax.set_ylim(0, 1.12)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
    ax.legend()
    fig.tight_layout()
    save(fig, "08b_eval_accuracy.png")

if len(eval_accuracies) > 1:
    acc_improvements = [eval_accuracies[i] - eval_accuracies[i-1]
                        for i in range(1, len(eval_accuracies))]
    acc_epoch_labels = [f"E{i}→E{i+1}" for i in range(1, len(eval_accuracies))]
    acc_colors = [BAR_GOOD if v > 0 else BAR_BAD for v in acc_improvements]

    fig, ax = plt.subplots(figsize=(8, 4))
    bars = ax.bar(acc_epoch_labels, acc_improvements, color=acc_colors,
                  width=0.5, edgecolor="#0f1117", linewidth=1.2)
    for bar, val in zip(bars, acc_improvements):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + (0.0005 if val >= 0 else -0.002),
            f"{val:+.4f}", ha="center",
            va="bottom" if val >= 0 else "top",
            fontsize=10, color="#ffffff", fontweight="bold",
        )
    ax.axhline(0, color="#ffffff", alpha=0.3, linewidth=1)
    ax.set_title("Eval Accuracy Improvement per Epoch (positive = improvement)",
                 fontsize=13, fontweight="bold", pad=12)
    ax.set_ylabel("Accuracy Gain")
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:+.2%}"))
    fig.tight_layout()
    save(fig, "09_accuracy_improvement.png")

n_plots = (
    2                               # train loss + eval loss
    + (1 if train_lrs       else 0)     # LR
    + (1 if train_gnorms    else 0)     # grad norm
    + 1                             # perplexity
    + 1                             # train accuracy
    + (1 if eval_accuracies else 0)     # eval accuracy (separate)
)
fig = plt.figure(figsize=(16, 4 * max(2, (n_plots + 1) // 2)), facecolor="#0f1117")
```

```
dashboard_title = (
    "MedGemma LoRA Fine-tuning Dashboard | "
    f"Model: {args.model} | LoRA r={args.lora_r} alpha={args.lora_alpha} | "
    f"LR={args.lr} | Epochs={args.epochs} | "
    f"Effective batch={args.batch_size * args.grad_accum}"
)
fig.suptitle(dashboard_title, fontsize=10, color="#e8eaf0", y=1.01)

gs = gridspec.GridSpec((n_plots + 1) // 2, 2, figure=fig,
                       hspace=0.45, wspace=0.3)
plot_idx = 0

def next_ax():
    nonlocal plot_idx
    row, col = divmod(plot_idx, 2)
    ax = fig.add_subplot(gs[row, col])
    plot_idx += 1
    return ax

# Train loss
ax = next_ax()
ax.plot(train_steps, train_losses, color=TRAIN_COLOR)
ax.set_title("Train Loss", fontsize=11)
ax.set_xlabel("Step", fontsize=9)

# Eval loss
if eval_losses:
    ax = next_ax()
    ax.plot(eval_epochs, eval_losses, color=EVAL_COLOR, marker="o", markersize=6)
    best_idx = eval_losses.index(min(eval_losses))
    ax.scatter([eval_epochs[best_idx]], [eval_losses[best_idx]],
               color="#ffd54f", s=80, zorder=5)
    ax.set_title("Eval Loss", fontsize=11)
    ax.set_xlabel("Epoch", fontsize=9)
    ax.set_xticks(eval_epochs)

# Learning rate
if train_lrs:
    ax = next_ax()
    ax.plot(train_steps[:len(train_lrs)], train_lrs, color=LR_COLOR)
    ax.set_title("Learning Rate", fontsize=11)
    ax.set_xlabel("Step", fontsize=9)
```

```
    ax.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))

# Grad norm
if train_gnorms:
    ax = next_ax()
    ax.plot(train_steps[:len(train_gnorms)], train_gnorms,
            color=GNORM_COLOR, alpha=0.8)
    ax.set_title("Gradient Norm", fontsize=11)
    ax.set_xlabel("Step", fontsize=9)

# Perplexity
ax = next_ax()
ax.plot(train_steps, train_ppl, color=PPL_COLOR, alpha=0.85, label="Train")
if eval_ppl:
    ax.plot([int(e * steps_per_epoch) for e in eval_epochs],
            eval_ppl, color=EVAL_COLOR, marker="o", markersize=6, label="Eval")
    ax.legend(fontsize=8)
ax.set_title("Perplexity", fontsize=11)
ax.set_xlabel("Step", fontsize=9)

# Train Accuracy (vs Step)
ax = next_ax()
ax.plot(train_steps, train_accuracies, color=ACC_COLOR, alpha=0.88)
ax.fill_between(train_steps, train_accuracies, alpha=0.12, color=ACC_COLOR)
ax.set_title("Train Accuracy (approx)", fontsize=11)
ax.set_xlabel("Step", fontsize=9)
ax.set_ylim(0, 1.05)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))

# Eval Accuracy (vs Epoch) — x-axis = epoch number, NOT steps
if eval_accuracies:
    ax = next_ax()
    ax.plot(eval_epochs, eval_accuracies, color=EVAL_COLOR, marker="o",
            markersize=7, linewidth=2.2)
    ax.fill_between(eval_epochs, eval_accuracies, alpha=0.12, color=EVAL_COLOR)
    best_acc_idx = eval_accuracies.index(max(eval_accuracies))
    ax.scatter([eval_epochs[best_acc_idx]], [eval_accuracies[best_acc_idx]],
               color="#ffd54f", s=90, zorder=5,
               label=f"Best: {eval_accuracies[best_acc_idx]:.1%}")
    ax.set_title("Eval Accuracy (approx)", fontsize=11)
    ax.set_xlabel("Epoch", fontsize=9)
    ax.set_xticks(eval_epochs)
```

```
        ax.set_ylim(0, 1.12)
        ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f"{y:.0%}"))
        ax.legend(fontsize=8)

    save(fig, "10_dashboard.png")

    log_path = out / "training_logs.json"
    with open(log_path, "w") as f:
        json.dump(log_history, f, indent=2)
    print(f"  Saved: {log_path}")

    print(f"  All graphs saved to: {out.resolve()}/")
    print(f"  Files:")
    for f in sorted(out.glob("*.png")):
        print(f"    {f.name}")

def main():
    args     = parse_args()
    hf_token = resolve_token(args.hf_token)
    random.seed(args.seed)
    np.random.seed(args.seed)

    # 0. Pin GPU — must happen before any CUDA allocations
    print("\n[0/4] Selecting GPU...")
    device = pin_gpu(args.gpu)
    torch.manual_seed(args.seed)

    print(f"\n{'='*60}")
    print(f"  Model    : {args.model}")
    print(f"  Output   : {args.output}")
    print(f"  Device   : {device}")
    print(f"  Epochs   : {args.epochs}")
    print(f"  Batch    : {args.batch_size} × {args.grad_accum} = "
          f"{args.batch_size * args.grad_accum} effective")
    print(f"  LR       : {args.lr}  |  LoRA r={args.lora_r} α={args.lora_alpha}")
    print(f"  Max len  : {args.max_len}")
    print(f"{'='*60}\n")

    # 1. Dataset
    print("[1/4] Loading dataset...")
    if args.dataset is None:
        args.dataset = autodetect_parquet()
```

```
    print(f"  Auto-detected: {args.dataset}")
elif not Path(args.dataset).exists():
    sys.exit("[ERROR] File not found: " + repr(args.dataset))
train_ds, val_ds = load_splits(args.dataset, args.val_split, args.seed)

# 2. Model — load in bfloat16, single device
print("\n[2/4] Loading model and processor...")
load_kw: Dict[str, Any] = {"dtype": torch.bfloat16}
if hf_token:
    load_kw["token"] = hf_token

try:
    processor = AutoProcessor.from_pretrained(args.model, **load_kw)
    model = AutoModelForImageTextToText.from_pretrained(
        args.model,
        device_map={"": 0} if device == "cuda" else None,  # force single GPU
        **load_kw,
    )
except Exception as e:
    sys.exit(
        f"[ERROR] Could not load model.\n  {e}\n\n"
        "  1. Accept model terms: huggingface.co/google/medgemma-1.5-4b-it\n"
        "  2. export HF_TOKEN=hf_...\n"
        "  3. Free up GPU memory — need ~8 GB for bfloat16\n"
        "     Check with: nvidia-smi"
    )

if processor.tokenizer.pad_token is None:
    processor.tokenizer.pad_token = processor.tokenizer.eos_token
    model.config.pad_token_id = model.config.eos_token_id
print("  → Model loaded in bfloat16")

# 3. LoRA
print("\n[3/4] Applying LoRA adapters...")
model.gradient_checkpointing_enable({"use_reentrant": False})
model.enable_input_require_grads()
model = apply_lora(model, args.lora_r, args.lora_alpha)

# 4. Train
print("\n[4/4] Starting training...")
out = Path(args.output)
out.mkdir(parents=True, exist_ok=True)
```

```
training_args = TrainingArguments(
    output_dir                  = str(out),
    num_train_epochs            = args.epochs,
    per_device_train_batch_size   = args.batch_size,
    per_device_eval_batch_size    = args.batch_size,
    gradient_accumulation_steps   = args.grad_accum,
    learning_rate               = args.lr,
    lr_scheduler_type           = "cosine",
    warmup_steps                = 5,
    bf16                        = (device == "cuda"),
    fp16                        = False,
    gradient_checkpointing       = True,
    gradient_checkpointing_kwargs = {"use_reentrant": False},
    logging_steps               = 10,
    eval_strategy               = "steps",
    eval_steps                  = 10,
    save_strategy               = "steps",
    load_best_model_at_end       = True,
    metric_for_best_model        = "eval_loss",
    greater_is_better           = False,
    save_total_limit            = 2,
    report_to                   = "none",
    dataloader_num_workers       = 0,
    remove_unused_columns        = False,
    seed                        = args.seed,
    push_to_hub                 = args.push_to_hub,
    hub_model_id                = args.hub_repo,
    hub_token                   = hf_token,
)

trainer = Trainer(
    model        = model,
    args         = training_args,
    train_dataset = train_ds,
    eval_dataset  = val_ds,
    data_collator = VQACollator(processor, args.max_len),
    callbacks     = [EarlyStoppingCallback(early_stopping_patience=2)],
)

trainer.train()
```

```
    # 5. Save
    print(f"\n✅ Saving model → {out}")
    trainer.save_model(str(out))
    save_processor_safe(processor, str(out))

    if args.push_to_hub and args.hub_repo:
        print(f"  Pushing to Hub: {args.hub_repo}")
        trainer.push_to_hub()

    logs = trainer.state.log_history
    train_losses = [l["loss"]      for l in logs if "loss"      in l]
    eval_losses  = [l["eval_loss"] for l in logs if "eval_loss" in l]
    print("\nTraining summary:")
    if train_losses: print(f"  Final train loss : {train_losses[-1]:.4f}")
    if eval_losses:  print(f"  Best eval loss   : {min(eval_losses):.4f}")
    print(f"  Saved to         : {out.resolve()}")

    print("\n📊 Generating training graphs...")
    try:
        plot_training_graphs(logs, str(out), args)
    except Exception as e:
        print(f"  ⚠ Graph generation failed: {e}")
        print("     Install matplotlib: pip install matplotlib")

if __name__ == "__main__":
    main()
```